BÖLÜM 5 / MODULE 5

Depolama, Güvenlik & Otomatik Ölçek-leme

Storage, Security & Horizontal Autoscaling

Kalıcı diskler (PV/PVC), StorageClass, RBAC erişim kontrolü, sıkı Pod güvenlik standartları, Namespace izolasyonu ve HPA ile otomatik ölçekleme.

Persistent storage (PV/PVC), StorageClasses, RBAC permissions, hardened Pod security contexts, Namespace tenancy, and HPA auto-scaling under load.

25

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

Kalıcı Depolama: PersistentVolume (PV) ve PersistentVolumeClaim (PVC)

Persistent Storage: PersistentVolumes (PV) and Claims (PVC)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Araba kiralama şirketi metaforu: 1) PersistentVolume (PV): Otoparkta bekleyen gerçek fiziksel arabadır (Sistem yöneticisinin hazırladığı 100GB'lık gerçek SSD disk). 2) PersistentVolumeClaim (PVC): Müşterinin elindeki kiralama kuponudur ('Bana 20GB'lık hızlı bir araba ver'). Kubernetes bu kuponu alır, otoparktaki uygun araba ile eşleştirir (Binding) ve anahtarı Spring Boot pod'una teslim eder!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The Car Rental Metaphor: 1) PersistentVolume (PV): The actual physical car parked in the garage (A real 100GB SSD provisioned by the infrastructure admin). 2) PersistentVolumeClaim (PVC): The customer's rental voucher ('I need a 20GB fast car'). Kubernetes matches the voucher to an available car (Binding) and hands the ignition keys to your Spring Boot Pod!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

PV küme çapında yönetilen gerçek depolama kaynağıdır (NFS, AWS EBS, Ceph). PVC ise bir pod'un talep ettiği depolama isteğidir (boyut ve erişim modu). Erişim modları: 1) `ReadWriteOnce` (RWO: Tek bir düğüm tarafından okunup yazılabilir; veritabanları için standarttır), 2) `ReadOnlyMany` (ROX: Birden fazla düğüm sadece okuyabilir), 3) `ReadWriteMany` (RWX: Birden fazla düğüm aynı anda yazabilir; NFS gerektirir). Pod PVC'yi bir volume olarak bağlar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

PV is a cluster-level resource backed by raw storage (AWS EBS, Ceph, NFS). PVC is a namespace-scoped ticket requesting capacity and access modes. Core modes: 1) `ReadWriteOnce` (RWO: mountable by a single node as read-write, ideal for databases), 2) `ReadOnlyMany` (ROX: concurrent read-only by multiple nodes), 3) `ReadWriteMany` (RWX: concurrent read-write, requires NFS/Gluster). K8s matches claims against volumes during the Binding lifecycle phase.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# coffee-pvc.yaml (Spring Boot Yüklenen Dosyalar İçin Kalıcı Disk Talebi)
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: coffee-receipts-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi

# Pod İçinde Bağlama (Volume Mount):
# spec.containers[0].volumeMounts:
#   - mountPath: /app/uploads
#     name: receipts-storage
# spec.volumes:
#   - name: receipts-storage
#     persistentVolumeClaim:
#       claimName: coffee-receipts-pvc
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

`ReadWriteOnce` modu 'aynı anda sadece 1 pod yazabilir' DEĞİL, 'aynı anda sadece 1 sunucu düğümü (Node) yazabilir' anlamına gelir! Farklı sunuculardaki iki pod aynı anda RWO diski paylaşamaz.

🇬🇧 WATCH OUT

`ReadWriteOnce` means mountable by a single NODE, not a single Pod! Pods residing on different physical nodes cannot attach to the same RWO volume simultaneously.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Pod silinse veya başka düğüme taşınsa bile PVC'deki veriler korunur.
- Yazılımcılar PV detaylarını bilmeden sadece PVC ile disk talep eder.
- Veritabanları ve kalıcı dosya yüklemeleri için zorunludur.

- Data persists safely even if the hosting pod is evicted or rescheduled elsewhere.
- Decouples application developers (PVC) from storage administrators (PV).
- Essential for reliable stateful workloads, file uploads, and databases.

26

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

StorageClass ve Dinamik Disk Tahsisi

StorageClasses and Dynamic On-Demand Provisioning

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Eskiden bir yazılımcı disk istediğinde sistem yöneticisine e-posta atar, yönetici veri merkezine gidip kablo takardı (Statik Tahsis – 3 gün sürer!). StorageClass ise otomat makinesidir: Parayı atarsın (PVC talebi gönderirsin), makine sana saniyeler içinde AWS veya Google Cloud'dan gıcır gıcır bir SSD disk üretip verir (Dinamik Tahsis)!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

In the old days, when a developer needed a disk, they filed a ticket and waited 3 days for a sysadmin to rack a physical hard drive (Static Provisioning). A StorageClass is a vending machine: drop your token (PVC manifest), and the machine auto-provisions a shiny new SSD volume directly from the cloud provider in 5 seconds (Dynamic Provisioning)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

StorageClass, arkasındaki CSI (Container Storage Interface) sürücüsünü (örn: `ebs.csi.aws.com`) ve disk parametrelerini (IOPS, disk tipi: gp3, io2) tanımlar. Bir PVC `storageClassName: fast-ssd` belirttiğinde, Kubernetes ilgili bulut sağlayıcısının API'sini çağırarak diski dinamik olarak oluşturur, anında bir PV nesnesi yaratır ve PVC'ye bağlar. `volumeBindingMode: WaitForFirstConsumer` ayarı diskin pod'un zamanlandığı uygun bölgede (Availability Zone) açılmasını sağlar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

StorageClasses abstract CSI (Container Storage Interface) provisioners (e.g. AWS EBS CSI, GCP PD CSI). When a PVC specifies a storageClass, the CSI driver automatically creates cloud block volumes on-demand without manual PV creation. Setting `volumeBindingMode: WaitForFirstConsumer` delays volume provisioning until pod scheduling resolves, avoiding cross-zone attachment failures.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# storage-class-fast.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-nvme-ssd
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer # Pod hangi AZ'ye düşerse diski orada yarat!
parameters:
  type: gp3
  iops: "3000"
allowVolumeExpansion: true # İleride diski küçültmeden büyütebilme izni!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Farklı Kullanılabilirlik Alanları (Availability Zones): AWS EBS diskleri başka bir AZ'deki sunucuya bağlanamaz! `WaitForFirstConsumer` kullanmazsanız disk A bölgesinde açılabilir ama pod B bölgesine düşüp asılı kalabilir.

🇬🇧 WATCH OUT

Cloud block volumes are pinned to Availability Zones! Omitting `WaitForFirstConsumer` can provision a disk in us-east-1a while the pod is scheduled in 1b, causing permanent mount hangs.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Dinamik tahsis manuel disk oluşturma ve bağlama yükünü sıfıra indirir.
- `allowVolumeExpansion: true` ile diskler canlıda kesintisiz büyütülebilir.
- CSI sürücüleri K8s ile modern bulut depolama sistemleri arasındaki standart köprüdür.

- Dynamic provisioning eliminates manual disk creation and operational tickets.
- Enables online live volume expansion without deleting claims or pods.
- CSI plugins standardize integration across public clouds and on-prem storage.

27

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

K8s Güvenliği & RBAC: ServiceAccount, Role ve RoleBinding

Kubernetes Security: RBAC (Roles, Bindings & ServiceAccounts)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Holding binası güvenlik protokolü: 1) Personel Kimlik Kartı (ServiceAccount): Uygulamanızın yakasına taktığı kimlik kartıdır. 2) Yetki Kılavuzu (Role): 'Sadece 2. kattaki çay ocağına girebilir, kasaya dokunamaz' kuralları. 3) Yetki Ataması (RoleBinding): O kimlik kartını o kurallara zimmetleyen resmi tutanaktır. Bu üçlü olmadan hiç kimse API Server'a adım atamaz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Skyscraper Security Protocol: 1) Badge (ServiceAccount): The digital ID card worn by your Spring Boot application. 2) Rulebook (Role): 'Allowed to visit the cafeteria on Floor 2, strictly prohibited from opening the executive vault'. 3) Assignment (RoleBinding): The signed HR form linking the Badge to the Rulebook. Without RBAC, nobody touches the API Server!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kubernetes RBAC (Role-Based Access Control) API Server'a gelen istekleri denetler. Bileşenler: 1) `Subject` (Kullanıcı veya ServiceAccount), 2) `API Groups` (, `apps`, `batch`), 3) `Resources` (`pods`, `services`, `configmaps`), 4) `Verbs` (`get`, `list`, `watch`, `create`, `delete`). `Role` sadece belirli bir namespace içinde geçerlidir; `ClusterRole` ise küme genelinde (örn: Node'ları veya tüm namespace'leri okumak için) geçerlidir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kubernetes RBAC authenticates and authorizes API requests. Every rule consists of: API Groups, Resources, and HTTP-like Verbs (`get`, `list`, `watch`, `create`, `delete`). A `Role` binds permissions strictly inside a namespace; a `ClusterRole` spans the entire cluster (e.g. cluster-wide node monitoring). ServiceAccount tokens are mounted into pods at `/var/run/secrets/kubernetes.io/serviceaccount`.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# Spring Cloud Kubernetes için Salt-Okunur İzin:
apiVersion: v1
kind: ServiceAccount
metadata:
  name: spring-cloud-sa
  namespace: coffee-shop
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: configmap-reader
  namespace: coffee-shop
rules:
  - apiGroups: [""]
    resources: ["configmaps", "pods"]
    verbs: ["get", "list", "watch"] # Sadece okuma izni ver!
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: bind-spring-reader
  namespace: coffee-shop
subjects:
  - kind: ServiceAccount
    name: spring-cloud-sa
roleRef:
  kind: Role
  name: configmap-reader
  apiGroup: rbac.authorization.k8s.io
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Uygulama pod'larına ASLA gereksiz yere `cluster-admin` yetkisi vermeyin! Spring Boot uygulamanızda bir güvenlik açığı (örn: Log4Shell) oluşursa, saldırgan tüm Kubernetes kümenizi ele geçirir (Least Privilege kuralı).

🇬🇧 WATCH OUT

NEVER grant `cluster-admin` to application pods! If an exploit strikes your Java code, the attacker inherits full administrative root over the entire cluster.

🎯 Özet Çıkarımlar

- En Az Ayrıcalık (Least Privilege) prensibiyle sadece gereken izinler verilmelidir.
- ServiceAccount uygulamaların K8s API'si ile güvenli konuşmasını sağlar.
- ClusterRole küme çapında, Role ise sadece tek bir namespace içinde geçerlidir.

🎯 Key Takeaways

- Always follow the Principle of Least Privilege: grant only essential verbs.
- ServiceAccounts provide secure identities for in-cluster API communication.
- ClusterRoles grant cluster-wide authority; Roles enforce namespace isolation.

28

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

Pod Güvenlik Standartları: SecurityContext & Non-Root

Hardening Pods: SecurityContext & Non-Root Execution

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Konteyneri çelik bir zırh içine kilitlemek! Bir korsan Spring Boot uygulamanızın içine sızmayı başarsa bile elini kolunu bağlamak istiyoruz: 1) 'Root değilsin, garibansın!' (`runAsNonRoot`), 2) 'Sunucudaki hiçbir dosyayı değiştiremezsin!' (`readOnlyRootFilesystem`), 3) 'Hiçbir ayrıcalık talep edemezsin!' (`allowPrivilegeEscalation: false`). Korsan içeri girse bile hiçbir şey yapamaz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Locking the container inside an iron cage! Even if a hacker breaches your Spring Boot app, you want them handcuffed: 1) 'You are not root; you are a powerless user!' (`runAsNonRoot`), 2) 'You cannot alter any system files!' (`readOnlyRootFilesystem`), 3) 'You cannot elevate permissions!' (`allowPrivilegeEscalation: false`). The hacker is trapped and helpless!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Pod Security Standards (PSS) üç düzey tanımlar: Privileged, Baseline ve Restricted. Üretim ortamında 'Restricted' hedeflenmelidir. `securityContext` bloğunda: 1) `runAsUser: 10001` (Root olmayan UID), 2) `allowPrivilegeEscalation: false` (setuid ikili dosyalarıyla yetki yükseltmeyi engeller), 3) `capabilities: drop: ['ALL']` (Linux çekirdek yeteneklerinin tamamını kaldırır), 4) `readOnlyRootFilesystem: true` (Konteynerin kök dosya sistemini salt-okunur yapar; yazılması gereken `/tmp` klasörü `emptyDir` ile ayrıca bağlanır).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Pod Security Standards (PSS) define Privileged, Baseline, and Restricted profiles. Secure production pods must adhere to Restricted profiles: 1) `runAsNonRoot: true` with explicit UID (10001), 2) `allowPrivilegeEscalation: false`, 3) `capabilities.drop: ['ALL']` stripping Linux kernel capabilities, 4) `readOnlyRootFilesystem: true` with an ephemeral `emptyDir` mounted at `/tmp` for JVM temp writes.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Zırhlı Pod Güvenlik Bloğu:
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    runAsGroup: 10001
    fsGroup: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: coffee-app
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop:
            - ALL
      volumeMounts:
        - mountPath: /tmp
          name: tmp-volume # Tomcat geçici dosyaları için geçici RAM diski
  volumes:
    - name: tmp-volume
      emptyDir: {}
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Spring Boot (özellikle gömülü Tomcat) `/tmp` klasörüne dosya yazmaya ihtiyaç duyar. Eğer `readOnlyRootFilesystem: true` yapıp `/tmp` klasörüne bir `emptyDir` bağlamazsanız uygulamanız açılışta `IOException` ile patlar!

🇬🇧 WATCH OUT

Embedded Tomcat crashes if it cannot write PID files or sockets to `/tmp`. When enabling `readOnlyRootFilesystem: true`, always mount an `emptyDir` to `/tmp`.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Konteyneri root olmayan kullanıcıyla çalıştırmak container-breakout saldırılarını önler.
- `readOnlyRootFilesystem` zararlı yazılımların ikili dosya indirmesini engeller.
- Linux yeteneklerinin (capabilities) tamamı varsayılan olarak düşürülmelidir.

- Non-root execution prevents catastrophic container breakout host compromises.
- Read-only root filesystems block malware from persisting payloads to disk.
- Dropping ALL Linux capabilities minimizes OS kernel attack surfaces.

29

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

Namespaces: Kümeyi Ortamlara Bölme (Dev, Staging, Prod)

Namespaces: Multi-Tenancy & Environment Isolation

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Dev bir alışveriş merkezini farklı mağazalara bölmek! Her mağaza (Namespace) kendi vitrinini düzenler. Bir stajyer 'Geliştirme' (Dev) mağazasındaki bir mankeni yanlışlıkla kırarsa, yan dükkandaki 'Üretim' (Production) kasasındaki paralar veya müşteriler bundan zerre kadar etkilenmez!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Partitioning a mega shopping mall into individual storefronts! Each store (Namespace) operates independently. If an intern accidentally knocks over a rack in the 'Dev' clothing shop, the real high-paying customers in the 'Production' jewelry store next door remain completely untouched and oblivious!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Namespace, fiziksel bir K8s kümesini mantıksal sanal kümelere böler. İsim çakışmalarını önler (farklı namespace'lerde aynı isimde `coffee-service` olabilir). İki kritik araçla desteklenir: 1) `ResourceQuota` (Namespace'in toplam CPU, RAM veya Pod sayısını sınırlar; örn: Dev namespace'i toplam en fazla 10 çekirdek CPU tüketebilir), 2) `LimitRange` (Pod başına varsayılan min/max kaynak sınırlarını zorunlu kılar).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Namespaces provide logical virtual partitioning within a single cluster. They prevent naming collisions and enforce governance via: 1) `ResourceQuota` (capping aggregate CPU, RAM, and object counts across the namespace), 2) `LimitRange` (enforcing default and min/max boundaries per container). Cluster-level resources (Nodes, StorageClasses, PVs) remain unnamespaced.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# dev-quota.yaml (Geliştirme Ortamı Kaynak Kotası)
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-compute-quota
  namespace: coffee-dev
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "10" # En fazla 10 pod açabilirler!

# Namespace oluştur ve aktif bağlamı değiştir:
# kubectl create namespace coffee-dev
# kubectl config set-context --current --namespace=coffee-dev
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Namespace'ler varsayılan olarak ağ düzeyinde yalıtım SAĞLAMAZ! Yani `coffee-dev` içindeki bir pod varsayılan ayarlarla `coffee-prod` içindeki bir pod'a ping atabilir. Tam izolasyon için mutlaka 23. Dersteki 'NetworkPolicy' kurallarını eklemelisiniz!

🇬🇧 WATCH OUT

Namespaces do NOT enforce network boundaries by default! Pods in `dev` can still ping pods in `prod` unless segmented with NetworkPolicies.

🎯 Özet Çıkarımlar

- Namespaces çok kiracılı (multi-tenant) ortamlarda düzen ve mantıksal sınır sağlar.
- ResourceQuota ekiplerin kümedeki tüm kaynakları sömürmesini engeller.
- `kubectl config set-context --current --namespace=...` ile varsayılan alan seçilir.

🎯 Key Takeaways

- Namespaces enable multi-tenant governance, organization, and name scoping.
- ResourceQuotas prevent rogue development teams from exhausting cluster capacity.
- Switch active context namespaces to avoid repetitive `-n <namespace>` flags.

30

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

Yatay Otomatik Ölçekleme (HPA - Horizontal Pod Autoscaler)

Horizontal Pod Autoscaling (HPA): Traffic-Driven Fleet Scaling

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

E-ticaret sitenizde çılgın Cuma indirimi başladı! Sabah 3 pod ile başlayan Spring Boot servisiniz, binlerce müşteri akın edince terlemeye başladı (CPU kullanımı %80'e fırladı). HPA uykusundan uyanır, anında pod sayısını 3'ten 10'a çıkarır! Gece yarısı kampanya bitip müşteriler uyuyunca, fazla pod'ları sessizce kapatıp tekrar 3'e düşürür. Cebinizi boşuna bulut faturası ödemekten kurtarır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Black Friday flash sale madness! Your coffee delivery backend starts sweating as CPU utilization spikes past 80%. The Horizontal Pod Autoscaler (HPA) wakes up and automatically multiplies your pod count from 3 to 10 in 60 seconds! When midnight strikes and traffic dies, HPA scales the fleet back down to 3, saving you thousands on cloud bills!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

HPA, Metrics Server'dan aldığı metrikleri (CPU, Memory veya özel Prometheus metrikleri) sürekli sorgular. Ölçekleme formülü basittir: `İstenen Pod Sayısı = ceil(Mevcut Pod Sayısı * (Mevcut Metrik Değeri / Hedef Metrik Değeri))`. HPA ani dalgalanmalarda sistemin sürekli açılıp kapanmasını (flapping/thrashing) önlemek için 'cooldown/stabilization windows' (varsayılan: küçülme için 5 dakika bekleme) uygular.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

HPA queries metrics from the Metrics Server or Custom Metrics APIs. Its scaling algorithm calculates: `desiredReplicas = ceil[currentReplicas * (currentMetricValue / desiredMetricValue)]`. To prevent flapping (rapid oscillations), HPA enforces stabilization cooldown windows (defaulting to 5 minutes for downscaling) ensuring stable, predictable autoscaling behavior.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# coffee-hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: coffee-hpa
  namespace: coffee-shop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: coffee-deployment
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70 # CPU %70'i geçerse yeni pod ekle!

# Yük testi yaparak HPA'yı canlı izle:
# kubectl get hpa -w
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

HPA'nın çalışabilmesi için Deployment manifestonuzda pod'lar için MUTLAKA `resources.requests.cpu` tanımlanmış olmalıdır! Request tanımlı değilse HPA CPU yüzdesini hesaplayamaz (`<unknown>/70%`) ve ölçekleme yapmaz.

🇬🇧 WATCH OUT

HPA requires `resources.requests` defined on container specs! Without requests, HPA cannot calculate percentage utilization and stays stuck in `<unknown>`.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- HPA trafik patlamalarında otomatik olarak yeni pod kopyaları açar.
- Sakin zamanlarda pod sayısını düşürerek bulut maliyetini minimize eder.
- Metrics Server kümede kurulu ve aktif olmalıdır.

- HPA automatically provisions extra replicas during unpredictable traffic spikes.
- Scales down automatically during idle periods to reduce operational infrastructure bills.
- Metrics Server must be deployed in the cluster to expose node and pod metrics.